

Formalization of Gödel’s Incompleteness Theorems in Basic Recursive Arithmetic (BRA)

Summary of an Agda development

May 2026

Abstract

We present a fully formalized proof of both Gödel’s First and Second Incompleteness Theorems in a tree-based variant of Guard’s Basic Recursive Arithmetic (BRA), executed in Agda 2.8 with `--safe --without-K --exact-split`. The development contains **zero postulates and zero holes**; `godelIII : Deriv Con -> Deriv bot` typechecks end-to-end against the abstract Goedel II skeleton.

There is a single `Term` datatype (no separate `Tree` datatype), and “codes” are exactly the value-shaped Terms certified by an `IsValue : Term -> Set` predicate (built only from `O` and `ap2 Pair`). The map `reify : Tree -> Term`, written explicitly throughout the codebase for clarity at code/term boundaries, is the identity.

The headline results are:

- $thm11 : Deriv\ G \rightarrow Deriv\ \perp$ (Gödel I);
- $godelIII : Deriv\ Con \rightarrow Deriv\ \perp$ (Gödel II).

The development follows Guard’s 1963 lecture notes [1]. The intermediate construction is a single first-order combinator `constr5 : Fun1` realising Guard’s “ $F(x)$ ” (page 17); each step is given by an explicit equational derivation.

A key conceptual point this note emphasises (Section 5) is that intermediate steps of Theorem 14’s chain produce *equational* `Derivs` of the form `atomic (eqn (thmT t) u)` where `u` is *not* `codeFormula P` for any $P : Formula$. Only the final `step5` conclusion has `u = codeFormula bot` (with $\perp$ genuinely closed). Guard’s chain operates one level lower than Hilbert–Bernays–Löb: instead of internal provability claims about specific formulas, it manipulates substituted-codes through standard BRA inference rules. This makes the entire proof internal and equational — a feature of the framework, not a gap.

1 Introduction

The aim is a feasible, ASCII-only, syntax-first Agda formalization of Gödel’s two incompleteness theorems following Guard’s 1963 system of *Basic Recursive Arithmetic* [1].

1.1 Single-layer Term language with *IsValue*

In BRA2 there is one syntactic layer.

The Term language.

- *Term*: built from `O` (leaf), `var n`, `ap1 f t`, `ap2 g t1 t2`.

- Function symbols: Fun_1 ($\{I, Fst, Snd, Comp, Comp_2, Z\}$, plus $KT_$ as a defined function), Fun_2 ($\{Pair, Const, Lift, Post, Fan, IfLf, TreeEq, treeRec\}$).
- *Formula*: $atomic (eqn \ell r)$, $not _$, $_imp _$.
- *Deriv F*: inductive family of derivations. Constructors for the algebraic axioms (ax_*), the equality axioms (Guard’s Ax 4–7), three propositional axioms (axK , axS , $axNeg$ in Lukasiewicz form), modus ponens (mp), substitution ($ruleInst$), and binary tree induction ($ruleIndBT$).

Codes are value-shaped Terms. “Codes” are Terms whose value-shape is certified by

$$IsValue : Term \rightarrow Set,$$

with constructors $valO : IsValue O$ and $valNd a b v_a v_b : IsValue (ap_2 Pair a b)$ (when $IsValue a$ and $IsValue b$).

The encoding maps now have type

$$code : Term \rightarrow Term, \quad codeFormula : Formula \rightarrow Term,$$

together with structural-soundness lemmas $code_isValue : (t : Term) \rightarrow IsValue (code t)$, $codeFormula_isValue$, $codeF_1_isValue$, $codeF_2_isValue$, and $natCode_isValue$ (BRA2/Term.agda, BRA2/Formula.agda). Every primitive constructor of an encoding produces a value-shaped Term.

Notational aliases Tree, lf, nd, reify. For continuity with Guard’s prose, BRA2/Term.agda exposes

$$Tree := Term, \quad lf := O, \quad nd a b := ap_2 Pair a b, \quad reify t := t.$$

The Layer-1/Layer-2 distinction is recorded as the $IsValue$ predicate; syntactically the layers are unified.

Reflection layer (object-level operations on codes). Functors that reason about BRA syntax internally:

- $cor : Fun_1$: defined as $Rec stepCor$ (BRA2/Cor.agda). Spec: $corOfReify : (t : Term) \rightarrow IsValue t \rightarrow Deriv (ap_1 cor t = code t)$. Operationally, cor takes a value-shaped Term (= a “numeral”) and returns its code.
- $sub : Fun_2$, $subT : Fun_2$: coded substitution (BRA2/Sub.agda, BRA2/SubT.agda), now indexed by $IsValue$ on the substituent.
- $thmT : Fun_1$: the proof-checker (Section 2).

Reflection theorem.

$$encode : (P : Formula) \rightarrow Deriv P \rightarrow \Sigma Term (\lambda y. \Sigma (IsValue y) (\lambda _. Deriv (thmT y = codeFormula P))).$$

The $IsValue y$ field certifies that every encoded proof tree is value-shaped, so $cor y$ provably reduces to $code y$ at the witness returned by $encode$. This certification is exactly what discharges the parametric $\mathbf{cor} \ x$ in the chain at the closure (see Section 5).

1.2 Notational convention: code-quotes and hats

Code-quotes $\ulcorner \cdot \urcorner$. We use $\ulcorner E \urcorner$ throughout for “the code (value-shaped Term) of E ”:

$$\ulcorner t \urcorner := \text{code } t \quad (t : \text{Term}), \quad \ulcorner F \urcorner := \text{codeFormula } F \quad (F : \text{Formula}).$$

Whenever a Guard-style expression mixes object-level and encoded material (e.g. $f(\underline{x})$), we keep the code-quote explicit: the formal object is the Term $\ulcorner f(\underline{x}) = \underline{f(x)} \urcorner$, never the unquoted equation. See Section 4.1 for the full caveat.

Hats $\hat{\cdot}$ and $\hat{\cdot}$. We write $\hat{x}$ as a shorthand for $\text{cor } x$, the numeral-encoding combinator applied to a Term x . This lets us write e.g. $\text{th } \hat{x}$ for $\text{ap}_1 \text{ thmT } (\text{ap}_1 \text{ cor } x)$ and $\text{sub } \hat{x} \hat{y}$ for $\text{ap}_2 \text{ sub } (\text{ap}_1 \text{ cor } x) (\text{ap}_1 \text{ cor } y)$.

For a Guard-style expression E , $\hat{E}$ denotes the *substituted-code* of E : the Term obtained by taking the code of E ’s pattern (with variable slots in place of $\hat{\cdot}$ -marked positions) and *sb*-substituting the $\hat{\cdot}$ -encoded numerals into those slots. E.g. $\widehat{\text{th } \hat{x}} = \text{ap}_2 \text{ Pair tagAp}_1 (\text{ap}_2 \text{ Pair } (\text{codeF}_1 \text{ thmT}) (\text{ap}_1 \text{ cor } x))$. At a free Term variable x , this substituted-code is in general not $\ulcorner P \urcorner$ for any $P : \text{Formula}$ (Section 5 discusses this in detail); for value-shaped x it BRA-reduces to a genuine codeFormula via *corOfReify*.

2 The provability predicate thmT

The Gödel-style “*th*” provability predicate is realized in BRA by a single Fun_1 -functor:

$$\text{thmT} = \text{Rec stepProtoWrapped} : \text{Fun}_1.$$

Here *stepProtoWrapped* is a Fun_2 assembled from a 40-deep linear *IfLf* cascade (BRA2/Thm/StepProto.agda and BRA2/Thm/ThmT.agda), one branch per primitive proof rule: ten equational axioms of Group I, eight Group II axioms, nine equality and propositional rules, and three rules of inference (*mp*, *ruleInst*, *ruleIndBT*).

The *key* property of thmT is the encoding-soundness lemma *encode* above, proved by induction on the *Deriv* derivation in BRA2/Thm/ThmT.agda’s *encodeRich*.

2.1 Self-substitution closure

thmT is a closed combinator:

$$\text{thClosed} : \forall x r, \quad \text{substF}_1 x r \text{ thmT} = \text{thmT}$$

and its encoding is var-free:

$$\text{codeF1Th_noVar} : \forall x \text{ repl}, \quad \text{codedSubst repl } (\text{code } (\text{var } x)) (\text{codeF}_1 \text{ thmT}) = \text{codeF}_1 \text{ thmT}.$$

3 Gödel’s First Incompleteness Theorem (Theorem 11)

3.1 The diagonal sentence

Following Guard, define

$$\begin{aligned} F_{\text{pre}} &:= \neg(\text{thmT } v_1 = \text{sub } v_0 v_0), \\ j &:= \text{codeFormula } F_{\text{pre}} \quad (\text{a closed value-shaped Term}), \\ G &:= F_{\text{pre}}[v_0 := j]. \end{aligned}$$

The sentence G has v_1 free; informally, G asserts of itself that no proof tree v_1 proves the formula whose code is $j[v_0 := j]$.

The certificate $j_isValue : IsValue\ j$ is exposed inside the `Diagonal` abstract block (`BRA2/Thm11Diagonal.agda`). It propagates through `Outer.GodelIII` and is what makes `corOfReify j j_isValue` usable downstream.

(For continuity with prior notation, the Agda codebase additionally defines `i := reify j`; since `reify` is the identity, $i = j$. We use j throughout this paper.)

3.2 Theorem 11

Theorem 1 (`thm11`, Gödel I). $thm11 : Deriv\ G \rightarrow Deriv\ \perp$, where $\perp := atomic\ (eqn\ trueT\ falseT)$, $trueT := O$, $falseT := ap_2\ Pair\ O\ O$.

Sketch. From any $\pi : Deriv\ G$ extract its encoding (y_d, v_d, E_1) where $v_d : IsValue\ y_d$ and $E_1 : Deriv\ (thmT\ y_d = codeFormula\ G)$. A separate diagonal lemma (`BRA2/Thm11Diagonal.agda`) yields $N : Deriv\ \neg(thmT\ y_d = codeFormula\ G)$, and $ax_{ExFalso} + two\ mp$'s collapse E_1 and N to $Deriv\ \perp$. $\square$

4 Gödel's Second Incompleteness Theorem (Theorem 14)

Theorem 2 (`godelIII`, Gödel II). $godelIII : Deriv\ Con \rightarrow Deriv\ \perp$, where $Con := \neg(thmT\ v_0 = codeFormula\ \perp)$.

4.1 Caveat: “ $f(\underline{x}) = \underline{f(x)}$ ” is not a BRA formula

A structurally crucial point, easy to miss in Guard's notation: the expression

$$f(\underline{x}) = \underline{f(x)}$$

is **not** a BRA formula. Guard's underline $(\underline{\cdot})$ takes a value and returns its numeral term — well-defined only at the meta-level for closed numerical inputs. For a variable x (the case Theorems 12–14 actually need), neither $\underline{x}$ nor $\underline{f(x)}$ has any BRA referent: there is no operator in BRA's term language that, given an arbitrary Term, returns “the numeral of its value.”

The only formal object that *does* correspond to Guard's expression is its *code*, the Term

$$\ulcorner f(\underline{x}) = \underline{f(x)} \urcorner,$$

which Definition 12 of Guard's notes *defines* by stipulation. In BRA2 this Term is built parametrically as $codeFTeq_1\ f\ x$, with $cor\ x$ filling the slot for the code of x 's numeral and $cor\ (f\ x)$ filling the slot for the code of $f(x)$'s numeral.

When x is a free Term variable, neither $cor\ x$ nor $cor\ (f\ x)$ reduces (the defining equation $corOfReify$ fires only on *IsValue* inputs), so $codeFTeq_1\ f\ x$ contains ap_1 -headed subterms literally. Codes (in the sense of $codeFormula\ P$) are *IsValue*-shape, so $codeFTeq_1\ f\ x$ at variable x is **not** of the form $codeFormula\ P$ for any $P : Formula$.

4.2 Theorem 12 — the encoded “ $th\ f$ ” clause

For every Fun_1 functor f , Theorem 12 produces a $Fun_1\ D_f$ such that $thmT$ evaluated at $D_f\ x$ is provably equal to the encoded form of “ $f(\underline{x}) = \underline{f(x)}$ ”:

Theorem 3 (*thm12_F1*). *thm12_F1* : $(f : \text{Fun}_1) \rightarrow \text{Thm12_F1_Result } f$ where *Thm12_F1_Result* *f* packages $D_f : \text{Fun}_1$ with *proof* : $(x : \text{Term}) \rightarrow \text{Deriv } (\text{thmT } (D_f x) = \text{codeFTeq}_1 f x)$.

The conclusion's RHS is the literal Pair-shape encoded equation. An analogous statement for *Fun2*. Structural recursion on *f* with one clause per constructor (15 total), BRA2/Thm12.agda module *FromBridges*.

4.3 Theorem 13 — under hypothesis $f x = y$

Lemma 4 (*thm13_F1*). For every $f : \text{Fun}_1$ and every Theorem 12 result r_{12} for *f*, *thm13_F1* *f* $r_{12} x y : \text{Deriv } (f x = y) \rightarrow \text{Deriv } (\text{thmT } (D_f x) = \text{codeFTeq}_{1,\text{Hyp}} f x y)$.

The conclusion replaces *cor* (*f* *x*) by *cor* *y*. One-line proof: *ruleTrans* on $r_{12}.\text{proof } x$ with *cong_R Pair* on *cong₁ cor* of the hypothesis.

4.4 Theorem 14 — the consistency-to-bot construction

The contract abstracted in BRA2/Thm14Abstract.agda asks for:

1. a *Fun*₁-functor *constr5* such that *thmT* (*constr5* *x*) syntactically encodes $\perp$;
2. a parametric internal-implication

$$\text{step5} : (x : \text{Term}) \rightarrow \text{Deriv}((\text{thmT } x = \text{codeFormula } G) \text{ imp } (\text{thmT } (\text{constr5 } x) = \text{codeFormula } \perp)).$$

The closure *closureToG* instantiates *step5* at v_1 , *ruleInsts Con*, modus-ponenses, transports through *subHeqcG* and G_{unfold} , arriving at *Deriv G*. Composing with *thm11* gives *godelIII*.

4.4.1 Construction of *constr5*

Two pre-existing tautologies, encoded:

- $t := (v_0 = v_2) \text{ imp } ((v_1 = v_2) \text{ imp } (v_0 = v_1))$ (transitivity);
- $t' := (v_0 = v_1) \text{ imp } ((\neg(v_0 = v_1)) \text{ imp } \perp)$ (ex falso).

Both BRA-derivable (BRA2/Thm14Numerals.agda); *encode* yields value-shaped Terms *tterm*, *t'term* certified by *t_term_isValue* / *t'_term_isValue* (these certificates are load-bearing for the chain — see Section 5).

The chain, paralleling Guard's page 17:

$$\begin{aligned} f_part(x) &:= \text{ruleInst } v_0 (th \hat{x}) (\text{ruleInst } v_1 (\text{sub } i \ i) (\text{ruleInst } v_2 \text{ cor } G \text{ tterm})), \\ g_part(x) &:= \text{mp}(\text{mp}(f_part(x), D_{\text{thmT}} x), D_{\text{sub}} i \ i), \\ K_part(x) &:= \text{ruleInst } v_1 (\text{cor } x) x, \\ h_part_skel(x) &:= \text{ruleInst } v_0 (th \hat{x}) (\text{ruleInst } v_1 (\text{sub } i \ i) \text{ t'term}), \\ \text{constr5}_{\text{final}}(x) &:= \text{mp}(\text{mp}(h_part_skel(x), g_part(x)), K_part(x)). \end{aligned}$$

4.4.2 The five steps

(BRA2/Th14*.agda.)

Step 1 — *step1_l* Theorem 13 applied to *thmT*.

Step 2 — *step2_l* Theorem 13 applied to *sub* at $(j, j, cor\ G)$ under closed hypothesis *subIleqcG*.

Step 3 — *step3_l* Three sb-applications on *t*, canonicalised, then two *mp* dispatches.

Step 4 — *K_part_l* One sb-application at v_1 on G_{norm} via the *PrAtX* *x* hypothesis to swap $thmT\ x \mapsto cor\ G$.

Step 5 — *step5_l* Two outer *mp* dispatches at $constr5_{\text{inner,final}}$ and $constr5_{\text{final}}$.

5 What is going on at the encoded layer: a remarkable internal proof

5.1 The phenomenon

The chain *step1_l*, ..., *step5_l* proves *Derivs* of the schematic form

$$Deriv\ (PrAtX\ x\ imp\ atomic\ (eqn\ (thmT\ t_i)\ u_i)),$$

where t_i, u_i are Terms depending on *x*. Several of these intermediate u_i are **not** of the form *codeFormula* *P* for any *P* : *Formula*.

A representative example: *step3_l* (BRA2/Th14Step3.agda) concludes

$$Deriv\ (PrAtX\ x\ imp\ atomic\ (eqn\ (thmT\ (g_part\ x))\ ap_2\ Pair\ (\widehat{th\ \hat{x}})\ \widehat{sub\ i\ i})),$$

with

$$\widehat{th\ \hat{x}} = ap_2\ Pair\ tagAp_1\ (ap_2\ Pair\ (codeF_1\ thmT)\ (ap_1\ cor\ x)).$$

At a variable *x* (e.g. $x = v_1$), $ap_1\ cor\ x$ does not reduce: *corOfReify* requires *IsValue* *x*, and v_1 is not value-shape. The Term $ap_1\ cor\ x$ has an ap_1 head; codes are exclusively *IsValue*-shape (built from *O* and $ap_2\ Pair$). Therefore $u_3 = ap_2\ Pair\ (\widehat{th\ \hat{x}})\ \widehat{sub\ i\ i}$ is not in the image of *codeFormula*.

The same phenomenon recurs in *step4*, *step5_intermediate*, *h_part_skel_l*. In each case, u_i is a *substituted-code* — the result of inserting *cor x* into a variable slot of the code of an open formula.

Yet *Deriv* (*atomic* (*eqn* *t u*)) is perfectly well-formed for arbitrary Terms *t, u* — the equational predicate *eqn* does not require *u* to be a code of a formula. The chain composes via the standard inference rules (*mp*, *ruleTrans*, *cong**, *axS*, *axK*) without ever needing the intermediate u_i to be *codeFormula* *P*.

5.2 Why this works: substituted-codes vs. codes-of-formulas

There are two distinct readings of Guard's notation $\ulcorner f(\underline{x}) \urcorner = \underline{f(x)}$:

- (A) **Code-of-substituted-formula.** Read $\underline{x}$ as *cor x* (a BRA Term), and form the BRA formula *atomic* (*eqn* ($ap_1\ f\ (cor\ x)$) ...). Take its *codeFormula*. The inner slot becomes *code* (*cor x*) (= the code of the BRA term *cor x*).
- (B) **Substituted-code.** Take the open formula *atomic* (*eqn* ($ap_1\ f\ v_n$) ...), code it, then *sb*-substitute *cor x* into the $\ulcorner v_n \urcorner$ slot. The inner slot becomes *cor x* itself.

At a free variable x these readings disagree: (A) places $\text{code } (\text{cor } x)$ in the slot, (B) places $\text{cor } x$. Theorem 12's codeFTeq_1 uses reading (B) — the substituted-code form. This is forced by the structure of D_f , which is built from $\text{Comp}_2 \text{ Pair } \dots \text{cor}$, producing $\text{cor } x$ in its evaluation directly.

The two readings coincide **when x is value-shaped**: corOfReify then gives $\text{cor } x = \text{code } x$, and *Lemma codeCommutesFormula* (BRA2/CodeCommutes.agda) gives

$$\text{code } (\text{substF } n \ t \ P) = \text{codedSubst } (\text{code } t) (\text{code } (\text{var } n)) (\text{codeFormula } P).$$

So at value-shape x , the substituted-code (B) is BRA-equal to $\text{codeFormula } (\text{substF } n \ x \ P_{\text{open}})$ — a genuine code-of-formula. At variable x this collapse fails, and (B) remains a hybrid Term.

5.3 The chain's discharge mechanism

Why does Goedel II close even though step 3 etc. produce hybrid RHS?

1. *Internal composition stays at the equational level.* The chain's $\text{mp}/\text{cong}/\text{ruleTrans}$ steps operate on $\text{eqn } t \ u$ for arbitrary t, u ; they do not require u to be $\text{codeFormula } P$. All intermediate Derivs are honest BRA derivations producing well-typed atomic equations.
2. *step5's conclusion is genuinely codeFormula $\perp$.* At step 5 the final RHS is $\text{codeFormula } \perp$, where $\perp$ is closed. No $\text{cor } x$ remains in the conclusion — the $\text{Comp2-of-KT-of-codeFormula-bot}$ construction in $\text{constr5}_{\text{final}}$ is a constant in x modulo Lift, so its thmT -image is a closed code by construction.
3. *Closure transports through notEqTransport at a value-shape witness.* closureToG instantiates step5 via ruleInst at the encode-witness of G 's proof. By the strengthened *encode* contract (returning IsValue), this witness is a value-shaped Term. At such a witness, $\text{cor } x$ inside any intermediate substituted-code rewrites to $\text{code } x$ via corOfReify , and the substituted-codes collapse to genuine codeFormulas of substituted formulas.

The interleaving — carry hybrid forms through internal-implication chains, collapse at the closure — is what lets the proof be parametric over $(x : \text{Term})$ rather than meta-iterated over closed numerals.

5.4 The projective specification of mp

A natural question raised by the chain is: what *exactly* is the specification of mp as embedded in thmT , given that step 3 applies mp -internalisation to inputs that are not $\text{codeFormula } (P \text{ imp } Q)$ for any $P, Q : \text{Formula}$?

A naive declarative reading would assert

$$\text{mp}(\text{codeFormula } (P \text{ imp } Q), \text{codeFormula } P) = \text{codeFormula } Q.$$

This is the soundness reading at well-typed inputs. But it is *not* what BRA's thmT actually computes, and it is not what step 3 uses. The actual specification is the verifying-body cascade in BRA2/SoundMpVProof.agda:

Given a coded proof tree $\text{arg} = \text{ap}_2 \text{ Pair } \text{tagMp } (\text{ap}_2 \text{ Pair } y_1 \ y_2)$, $\text{thmT}(\text{arg})$ dispatches via $\text{body_mp_v}(a, b) = \text{Post verifierF}_1 \text{ Pair } a \ b$ where verifierF_1 first computes $\text{thmT } y_1$ and $\text{thmT } y_2$, then performs *two TreeEq* checks:

1. *outerCheck*: $Fst(thmT\ y_1) \stackrel{?}{=} tagImp$,
2. *innerCheck*: $thmT\ y_2 \stackrel{?}{=} Fst(Snd(thmT\ y_1))$.

If both succeed, the verifier returns $Snd(Snd(thmT\ y_1))$, the consequent slot. If either fails, it returns $codeTriv = falseT$, the safe default.

Soundness. The two *TreeEq* checks together ensure that the consequent extracted is the consequent of *this specific implication* applied to *this specific antecedent*. Without the inner check $thmT\ y_2 = Fst(Snd(thmT\ y_1))$, BRA’s *mp*-dispatch would project the second slot of an arbitrary *tagImp*-headed Term, with no relation to y_2 , and *thmT* would fail to be a sound proof-checker (the encoding-soundness lemma in BRA2/Thm/ThmT.agda’s *encodeRich* would not go through). The inner check is what makes the *mp* clause of *encodeRich* valid: it certifies, equationally inside BRA, that the two sub-proofs y_1, y_2 are compatible at the expected antecedent.

Why this still works on substituted-codes. The two *TreeEq* checks are *equational comparisons on Terms*, not type-checks on Formulas. Whatever sits in the inner slots of $thmT\ y_1$ and $thmT\ y_2$ — *codeFormula P* and *codeFormula Q*, or *cor x* interleaved with tag constants, or any value-shaped Term — only matters in that the inner check fires correctly: the antecedent slot of $thmT\ y_1$ must be *TreeEq*-equal to $thmT\ y_2$.

In step 3 of Theorem 14, the equational chain is set up precisely so that this match holds: each *mp*-internalised step ensures, via the previous step’s *Deriv*, that the antecedent slot of the current implication’s substituted-code is exactly the consequent of the previous step. So the two *TreeEq* checks both fire *equationally*, with both sides being substituted-codes, and the dispatch returns the consequent’s substituted-code. The chain composes precisely because both readings of *mp* — the “well-typed Formula” and “well-shaped substituted-code” cases — are handled by the same projective primitive-recursive computation.

The result is again a substituted-code, fed into the next step the same way:

$mp\text{-}body(\text{substituted-code-of-implication}, \text{substituted-code-of-antecedent}) = \text{substituted-code-of-consequent}$,

equationally, with no claim about Formula content but with the soundness-relevant antecedent-match certified by *TreeEq*.

The two readings of *mp*’s specification stand in a clear hierarchy:

Projective (what BRA actually computes). The *Fst/ Snd* projection above, parametric in the inner Terms. Total on all Term inputs of the right outer shape, producing a Term output by syntactic projection.

Declarative (what soundness justifies). At literal *codeFormula (P imp Q)* inputs, the projective behaviour delivers *codeFormula Q*. This is what makes *thmT* a sound proof-checker (*encodeRich*’s *mp* clause).

The soundness reading factors through the projective reading: BRA’s *mp* primitive only knows about Term shape, and Formula-content correctness is the *consequence* for the well-shaped subset of inputs. Theorem 14 step 3 uses the projective reading directly; the final closure converts to the declarative reading by instantiating at a value-shape witness.

The same observation lifts to the other “soundified” rule dispatchers in the chain (*ruleSym*, *cong₁*, *cong_L*, *cong_R*, *ruleTrans*, *ruleInst*, *ruleInst₂*, *ruleIndBT*). Each is specified as a primitive-recursive Term projection on a shape-discriminated input; each delivers a Formula-content interpretation only on its well-shaped subset. The verifying-body construction ($body_X_v = Post\ verifierX\ Pair$,

Section “Sound *thmT*” of `README.md`) is precisely what enforces the safe default *codeTriv* on malformed inputs without spoiling the projective behaviour on well-shaped ones.

In one line: *BRA’s mp is a Term projection that happens to internalise Formula modus ponens at well-typed inputs.* The chain in step 3 uses the projection-on-Term reading; the internal-modus-ponens reading is what justifies it semantically at the value-shape closure. Both readings are valid specifications; the projective one is strictly more general and is what makes the chain compose at variable x .

5.5 Comparison with Hilbert–Bernays–Löb

Standard textbook Gödel II proofs go through provability statements:

$$D1 : \vdash A \implies \vdash \text{Pr}(\ulcorner A \urcorner), \quad D2 : \vdash \text{Pr}(\ulcorner A \rightarrow B \urcorner) \rightarrow \text{Pr}(\ulcorner A \urcorner) \rightarrow \text{Pr}(\ulcorner B \urcorner), \quad D3 : \vdash \text{Pr}(\ulcorner A \urcorner) \rightarrow \text{Pr}(\ulcorner \text{Pr}(\ulcorner A \urcorner) \urcorner).$$

Every premise is a provability statement *about* a code of a specific formula. The chain stays at the level “ F is provable” for various F . Reflection (D3) is needed to handle the iteration.

Guard’s BRA chain operates one level lower: at the level of *equations* *thmT* $t = u$ for arbitrary Terms t, u . Reflection (D3-analogue) is realised internally and parametrically, without reference to any specific formula:

- Theorem 12 is the parametric “ D_f is the encoded image of f ” — one statement, all f .
- Theorem 13 lifts external equations to encoded form, parametric in the equational hypothesis.
- Theorem 14 chains substituted-codes through *mp/ sb/ruleInst*, never invoking a separate “ $\text{Pr}(\ulcorner \text{Pr}(\ulcorner G \urcorner) \urcorner)$ ” reflection axiom. *step5* is an equational consequence of Theorem 12’s defining equations under hypothesis, not a provability claim about a specific formula.

5.6 Remarkable consequences

1. **Avoids meta-reflection.** No $\text{Pr}(\ulcorner \text{Pr}(\ulcorner F \urcorner) \urcorner)$ axiom is invoked. The analogous internalisation falls out of *thmT*’s defining equations + *Comp2/KT/cor* compositions.
2. **Fully equational.** Goedel II reduces to BRA equational reasoning + *mp/axS/axK/axNeg/axContrapos.* No specialised “provability logic” is needed.
3. **Parametric over** ($x : \text{Term}$). Each step lemma works for any Term x (variable, application of *cor* to a variable, anything). Standard HBL proofs are usually meta-iterated over each closed witness.
4. **The intermediate Derivs are genuine BRA proofs that do not correspond to “BRA proves formula F ” for any single F .** This is the structurally interesting feature: the chain proves *equational* statements between Terms, with the “provability” interpretation recovered only at the closure.

The cost is bookkeeping that the development makes explicit: the substituted-code RHS at variable x requires *IsValue* certificates (on j , on encode-witnesses, on t_term/t'_term) propagated through to the closure. These certificates are the formal counterpart of Guard’s silent assumption that “ x ” for variable x is sensible because x will eventually be instantiated at a closed numeral.

In short: *Goedel II in BRA is an equational proof at the code layer, internalised via primitive recursion, with the provability-logic story emerging only at closure.* Guard’s 1963 formulation has this character intrinsically; HBL-style proofs do not. The Agda dev makes the distinction visible.

6 Methodology

6.1 Constraints maintained throughout

- `--safe --without-K --exact-split` on every file.
- ASCII only. No Unicode.
- No postulates. No holes. No `with`-abstraction. No dot patterns (one exception: `.0 / .a .b` in *IsValue* pattern matches, which are the canonical syntax for matching against an indexed family's index).
- No new *Deriv* axioms beyond Guard's primitive list.

6.2 Verification

```
$ agda --safe BRA2/GoedelIIFull.agda
$ echo $?
0
$ ls BRA2/GoedelIIFull.agdai
BRA2/GoedelIIFull.agdai
$ grep -rn "^postulate" BRA2/
$ # (empty)
```

7 Discussion

The two Gödel theorems in BRA2:

- **Gödel I.** If the diagonal sentence G is provable, then $\perp$ is provable.
- **Gödel II.** If the consistency formula Con is provable, then $\perp$ is provable.

The intermediate functions $constr5_{\text{final}}$ and the parametric step lemmas $step1_l$ – $step5_l$, K_part_l , $h_part_skel_l$ are *first-order* in the sense of recursive-arithmetic definability: each is a closed combinator built from primitive Fun_1 / Fun_2 symbols, and each *Deriv* derivation is given by an explicit equational proof in the BRA system. No external soundness theorem is invoked.

7.1 Comparison with prior formalisations

The only previous machine formalisation of Gödel's *Second* Incompleteness Theorem we are aware of is Paulson's Isabelle/HOL development of hereditarily finite set theory [2], following Świerczkowski [3]. Other formalisations (O'Connor in Coq, Han in Lean, Carneiro's metamath work) target Gödel's *First* theorem only.

Paulson's development uses HF set theory and full Gödel numbering of Hilbert-style proofs, with encoded substitution treated as an external soundness theorem. Bound variables in the formal language (HF has $\exists$ and $\forall$ as primitive binders, with bound variables significant for inductive proofs over formula structure) are handled via *Nominal Isabelle* [5]: a non-standard extension of Isabelle/HOL that quotients binding constructions modulo α -equivalence, with permutations on names as the underlying mechanism. Paulson reports that the nominal framework is essential to make induction over formula structure tractable (O'Connor's first-incompleteness Coq formalisation needed roughly 1900 lines for a single substitution lemma without nominal support); the cost is

that any function defined on formulas must be proved to use bound variables “sensibly,” a property whose proofs are demanding and required dedicated assistance from Christian Urban. For coding, Paulson separately uses de Bruijn indices and proves a correspondence with the nominal syntax.

The present development sidesteps this entire issue. BRA’s *Formula* type has no quantifiers — only atomic equations, $\neg$, and $\supset$ — so there are no bound variables in the formal language at all; what appears in *atomic* (*eqn* ℓ r) are just object-level Terms, with free variables denoted *var* n for $n : \mathbb{N}$. Universal closure of an open formula is expressed implicitly via the BRA rule *ruleInst* (substitution into a derived formula), without any meta-level renaming. Substitution at the term level is the ordinary primitive-recursive *subst* : $\mathbb{N} \rightarrow \text{Term} \rightarrow \text{Term} \rightarrow \text{Term}$, capture-free by construction (no binder under which to capture). Coded substitution *sub* : *Fun*₂ is itself a BRA functor; its correctness is the equational *codeCommutatesFormula* lemma, not a metatheoretic substitution-lemma proof. In short, what required Nominal Isabelle in Paulson’s HF formalisation is for us a non-issue, because Guard’s BRA was designed without binding constructors in the first place.

Paulson’s “mistake” in Boolos–Świerczkowski, and Guard’s analogous construction.

Paulson devotes §6 of his paper to identifying a presentational flaw in Boolos [4] and Świerczkowski’s expositions of the second incompleteness theorem in HF. Both sources describe the internal substitution as $\text{Pf}[\alpha]_V(Q)$, read informally as “ α with each free variable x_i replaced by the formal term $Q(x_i)$,” where Q is a “pseudo-function” mapping a value to its name in the formal language. Paulson’s critique: Q is not a function of the formal language (HF has only one function symbol, $\triangleleft$), and any version of the substitution that leaves the original variables x free in the transformed formula does not work — because x ranges over all HF sets, including codes of formulas, and the supposed equation $Q(x) =$ “the numeral of x ” has no formal status. Paulson’s corrected version introduces *new* free variables x' that appear in the transformed formula, constrained by relational antecedents $\text{QR}(x, x')$ on the left of the turnstile. In §7 he further shows that Świerczkowski’s earlier attempt to define such a Q via a total ordering on the HF universe has a significant gap, which Paulson closes by working with the relation QR throughout instead of the function Q .

The same conceptual issue arises in Guard’s notation in `guard15.pdf`. Guard writes the encoded equational defining clause for a primitive functor f as

$$\text{th}(D_f x) = \ulcorner f(\underline{x}) = \underline{f(x)} \urcorner,$$

where the underline $(\underline{\cdot})$ takes a value to its numeral term — exactly Boolos–Świerczkowski’s pseudo-function Q at a different name. At a free variable x this notation is not a BRA expression: there is no operator in BRA’s term language that, given an arbitrary Term, returns “the numeral of its value” (Section 4.1). Read literally, Guard’s $\underline{x}$ would be the same kind of pseudo-function Paulson criticises.

The BRA chain treats this rigorously the way Paulson treats his QR . The combinator *cor* : *Fun*₁ is a genuine BRA functor (Layer 3, BRA2/Cor.agda) with a primitive recursive defining equation; for value-shaped inputs x it satisfies $\text{ap}_1 \text{ cor } x = \text{code } x$ internally (the lemma *corOfReify*). At free variables it does *not* reduce, just as Paulson’s $\text{QR}(x, x')$ does not equate x with anything specific: instead it constrains x' relationally. Theorem 14 step 3 in our development then proves internal-implication statements

$$\text{thm } T x = \text{codeFormula } G \rightarrow \text{thm } T (g_part x) = \text{th } \widehat{\hat{x} = \text{sub } j j},$$

parametric in x , with the $\hat{x} = \text{cor } x$ slot left syntactic and the closure transporting it to *code* x only at the value-shaped instantiation step (the encode-witness, certified by *IsValue*). This is

structurally the same move as Paulson’s: introduce parametric new variables, constrain by relation on the antecedent side, never assert “ $\hat{x}$ equals the numeral of x ” as an unconditional formal claim.

A further connection to Paulson’s §7: Świerczkowski needed an ordering on the HF universe to define Q by recursion on rank, and the resulting construction had a gap. In Guard’s BRA, *cor* requires no such ordering — it is built directly as *Rec stepCor* from BRA’s primitive-recursion combinator on binary trees (BRA2/Cor.agda), with *stepCor* a compact $Fan + Lift + KT + Post + Pair$ expression. The primitive-recursive structure of trees is enough; no recursion on rank, no ordering, no gap.

The present development follows Guard [1]: the “provability predicate” *thmT* is itself a primitive-recursive combinator built from $Rec + Fan + Lift + IfLf$; encoded substitution is performed by the explicit primitive recursive functor $sub : Fun_2$; the entire chain stays first-order; and the proof is equational at the encoded layer rather than provability-logical at the formula layer (Section 5).

References

- [1] J. R. Guard, *Lecture notes on recursive arithmetic*, Applied Mathematics Division, Argonne National Laboratory, 1963.
- [2] L. C. Paulson, *A machine-assisted proof of Gödel’s incompleteness theorems for the theory of hereditarily finite sets*, Review of Symbolic Logic, vol. 7 no. 3, 2014, pp. 484–498.
- [3] S. Świerczkowski, *Finite sets and Gödel’s incompleteness theorems*, Dissertationes Mathematicae, vol. 422, Polish Academy of Sciences, 2003.
- [4] G. S. Boolos, *The Logic of Provability*, Cambridge University Press, 1993.
- [5] C. Urban and C. Kaliszyk, *General bindings and alpha-equivalence in Nominal Isabelle*, Logical Methods in Computer Science, vol. 8 no. 2, 2012.
- [6] R. Davies and F. Pfenning, *A modal analysis of staged computation*, Journal of the ACM, vol. 48 no. 3, May 2001, pp. 555–604.